

אימות תכונות ω -רגולריות

גרא וייס

המחלקה למדעי המחשב

אוניברסיטת בן-גוריון





ראשי פרקים

תזכורת: בדיקה תכונות בטיחות רגולריות

בדיקת תכונות ω -רגולריות

בדיקת תכונות התמדה

רדוקציה לתכונות התמדה

תכונות התמדה (persistence)

חיפוש DFS מקונן

סיכום תכונות רגולריות



תזכורת: אלגוריתם לבדיקת תכונות בטיחות רגולריות

$\mathcal{A}$ הוא אוטומט כך ש- $\mathcal{L}(\mathcal{A}) = \text{BadPref}(P)$

$$TS \models P_{\text{safe}} \Leftrightarrow \text{Traces}_{\text{fin}}(TS) \cap \text{BadPref}(P_{\text{safe}}) = \emptyset$$

$$\Leftrightarrow \text{Traces}_{\text{fin}}(TS) \cap \mathcal{L}(\mathcal{A}) = \emptyset$$

$$\Leftrightarrow TS \times \mathcal{A} \models \underbrace{\text{Always}(\bigwedge_{q \in F} \neg q)}_{\text{תכונת שמונה}}$$

F היא קבוצת
המצבים
המקבלים באוטומט

מסקנה: בדיקת תכונות בטיחות רגולרית שקולה לבדיקת שמונה על $TS \times \mathcal{A}$

משמעות: ניתן לבדוק תכונות בטיחות רגולרית באמצעות DFS או BFS



תזכורת: תכונה ω -רגולריות

תכונת זמן-ליניארי P נקראת ω -רגולרית

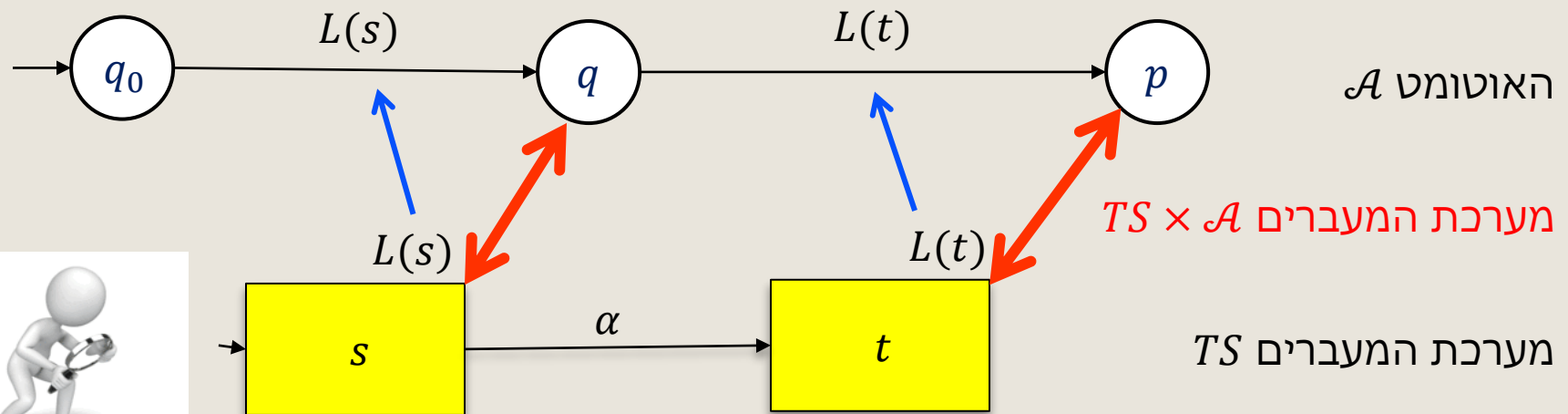
אם

P היא שפה ω -רגולרית (קיים $\mathcal{A}$ כך ש- $P = \mathcal{L}_\omega(\mathcal{A})$)

$$TS \stackrel{?}{=} \mathcal{L}_\omega(\mathcal{A})$$

איך נבדוק אם מערכת מעברים מקיימת תכונה ω -רגולרית?

תזכורת: הרכבה של מערכת מעברים ואוטומט



כל מילה ב- $Traces(TS \times \mathcal{A})$ מייצגת ריצה של המערכת וריצה מסוימת של האוטומט על עקבות הריצה. כיוון שיכולות להיות כמה ריצות של האוטומט על אותה המילה (הוא לא בהכרח דטרמיניסטי), יכולים להיות כמה רצפי עקבות ב- $Traces(TS \times \mathcal{A})$ לאותה הריצה של TS . יכול להיות שחלקם יהיו "מקבלים" וחלקם לא.



ראשי פרקים

בדיקה תכונות בטיחות רגולריות

בדיקת תכונות ω -רגולריות

בדיקת תכונות התמדה

רדוקציה לתכונות התמדה

תכונות התמדה
(persistence)

חיפוש DFS מקונן

סיכום תכונות רגולריות

רעיון האלגוריתם לבדיקת תכונות ω -רגולריות

$\mathcal{A}$ הוא אוטומט כך ש- $\mathcal{L}_\omega(\mathcal{A}) = ((2^{AP})^\omega \setminus P)$

$$TS \models P \iff \text{Traces}(TS) \subseteq P$$

$$\iff \text{Traces}(TS) \cap ((2^{AP})^\omega \setminus P) = \emptyset$$

$$\iff \text{Traces}(TS) \cap \mathcal{L}_\omega(\mathcal{A}) = \emptyset$$

$$\iff TS \times \mathcal{A} \models \underbrace{\{\sigma: \exists j \geq 0 . \forall i \geq j . \overbrace{\sigma[i] \cap F = \emptyset}^{\text{תנאי התמדה}}\}}_{\text{תכונת התמדה}}$$

מסקנה: בדיקת תכונה ω -רגולרית על TS
שקולה לבדיקת תכונת התמדה על $TS \times \mathcal{A}$

סטודנט תוהה למה לא השתמשנו באוטומט כך ש- $\mathcal{L}_\omega(\mathcal{A}) = P$



רעיון שגוי לבדיקת תכונות ω -רגולריות

$\mathcal{A}$ הוא אוטומט כך ש- $\mathcal{L}_\omega(\mathcal{A}) = P$

$$TS \models P \iff \text{Traces}(TS) \subseteq P$$

$$\iff \text{Traces}(TS) \cap ((2^{AP})^\omega \setminus P) = \emptyset$$

$$\iff \text{Traces}(TS) \cap ((2^{AP})^\omega \setminus \mathcal{L}_\omega(\mathcal{A})) = \emptyset$$

$$\nRightarrow TS \times \mathcal{A} \models \{\sigma: \forall j \geq 0, \exists i \geq j \text{ such that } \sigma[i] \cap F \neq \emptyset\}$$



בנו דוגמא נגדית

מה אפשר להניח על $\mathcal{A}$ כדי שהגרירה תהיה נכונה?

למה הגרירה הזאת לא נכונה?



מסקנה: עדיף להשתמש ברעיון שהוצג בשקף הקודם ☺

תכונת התמדה

Persistence Property



תכונת התמדה מעל AP היא תכונת זמן ליניארי $P_{pers}(\Phi)$
 מהצורה "בסופו של דבר מתקיים Φ לעד" עבור תנאי מצב Φ

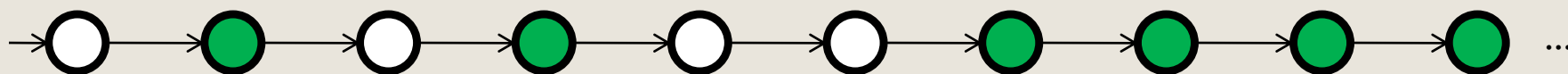
$$P_{pers}(\Phi) = \{\sigma: \exists i \geq 0 . \forall j \geq i . \sigma[j] \models \Phi\}$$



האם קיימת תכונה
 שהיא גם תכונת התמדה
 וגם תכונת שמורה?

Φ נקרא תנאי ההתמדה של $P_{pers}(\Phi)$

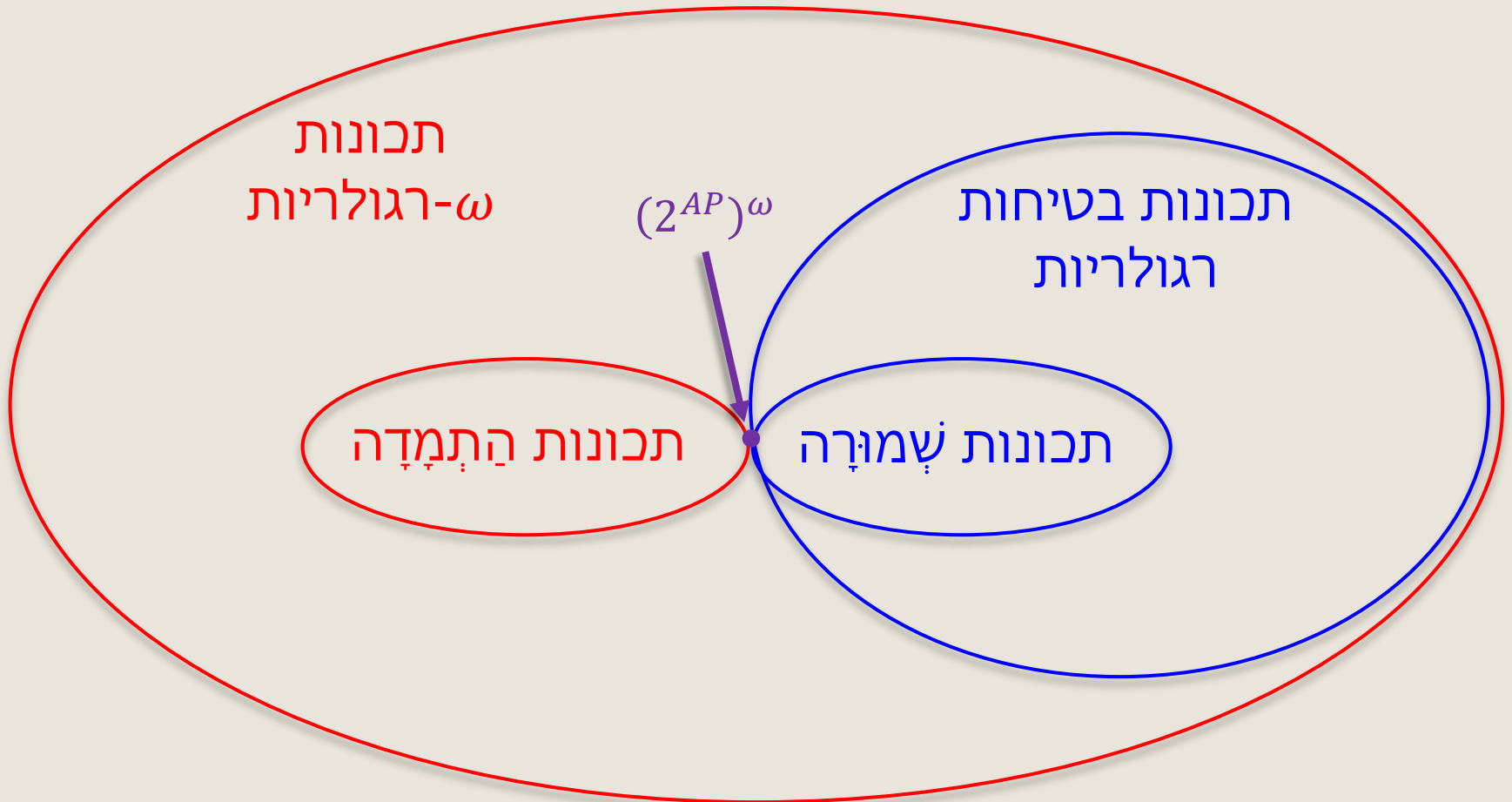
"אחרי רישא סופית, Φ מתקיימת כל הזמן"



Eventually(Always Φ)



תכונות התמדה הן לא תכונות שמונה



תכונות התמדה ימלאו, עבור תכונות ω -רגולריות, את התפקיד שמילאו תכונות שמונה עבור תכונות בטיחות רגולריות



בדיקת התמדה

- מטרה: לקבוע אם $TS \models P_{pers}(\Phi)$
- אם יש מצב נגיש s ב- TS כך ש- $\Phi \neq s$ ו- s נמצא על מעגל



- בגלל ש- s נגיש, קיים מקטע מסלול התחלתי המסתיים ב- s
- בגלל ש- s נמצא על מעגל, המקטע יכול להמשיך למקטע אינסופי
- TS עלולה לבקר במצב s אינסוף פעמים ולכן $TS \models P_{pers}(\Phi)$
- אם לא קיים כזה s אז $TS \models P_{pers}(\Phi)$



תזכורת: מערכת מעברים הנוצרת מהרכבת אוטומט עם מערכת מעברים

עבור מערכת מעברים $TS = \langle S, Act, \rightarrow, I, AP, L \rangle$ ואוטומט $\mathcal{A} = \langle Q, \Sigma, \delta, Q_0, F \rangle$
כך ש- $\Sigma = 2^{AP}$ ו- $Q_0 \cap F = \emptyset$ הגדרנו מערכת מעברים:

$$TS \times \mathcal{A} = \langle S \times Q, Act, \rightarrow_{\times}, I_{\times}, AP_{\times}, L_{\times} \rangle$$

באשר $AP_{\times} = Q$, $L_{\times}(\langle s, q \rangle) = \{q\}$, ויחס המעברים $\rightarrow_{\times}$ מוגדר ע"י

$$\frac{s \xrightarrow{\alpha} t \wedge p \in \delta(q, L(t))}{\langle s, q \rangle \xrightarrow{\alpha}_{\times} \langle t, p \rangle}$$

וקבוצת מצבי ההתחלה $I_{\times}$ מוגדרת ע"י

$$I_{\times} = \{ \langle s_0, q \rangle : s_0 \in I \wedge \exists q_0 \in Q_0 . q \in \delta(q_0, L(s_0)) \}$$



משפט: אימות תכונות ω -רגולרית

עבור:

- TS מערכת מעברים בלי מצבים סופיים מעל AP
- P תכונה ω -רגולרית
- $\mathcal{A}$ אוטומט בלתי חוסם כך ש- $\mathcal{L}_\omega(\mathcal{A}) = (2^{AP})^\omega \setminus P$

התכונות הבאות שקולות:

- 1) $TS \models P$
- 2) $\text{Traces}(TS) \cap \mathcal{L}_\omega(\mathcal{A}) = \emptyset$
- 3) $TS \times \mathcal{A} \models P_{\text{pers}(\sigma \cap F = \emptyset)}$

מסקנה: בדיקת תכונה ω -רגולרית שקולה לבדיקת תכונת התמדה על $TS \times \mathcal{A}$



הוכחה (כיוון ראשון)

$\pi' = \langle s_0, q_1 \rangle, \langle s_1, q_2 \rangle, \dots \in \text{Paths}(TS \times \mathcal{A})$ such that $\pi' \not\models P_{\text{pers}(\mathcal{A})}$

$\pi = s_0, s_1, \dots$ is a path in TS

$\exists^\infty .q_i \in F$

Let $q_0 \in Q_0$ such that $q_0 \xrightarrow{L(s_0)} q_1$

$q_0, q_1, \dots$ is a run in $\mathcal{A}$ for $\text{trace}(\pi) = L(s_0)L(s_1)\cdots \in \text{Traces}(TS)$

$\exists^\infty .q_i \in F \implies q_0, q_1, \dots$ is accepting $\implies \text{trace}(\pi) \in \mathcal{L}_\omega(\mathcal{A})$

$\text{Traces}(TS) \cap \mathcal{L}_\omega(\mathcal{A}) \neq \emptyset$



הוכחה (כיוון שני)

$$\text{Traces}(TS) \cap \mathcal{L}_\omega(\mathcal{A}) \neq \emptyset$$

$$\exists \pi = s_0, s_1, \dots \in \text{Paths}(TS) \text{ with } \text{trace}(\pi) = L(s_0)L(s_1), \dots \in \mathcal{L}_\omega(\mathcal{A})$$

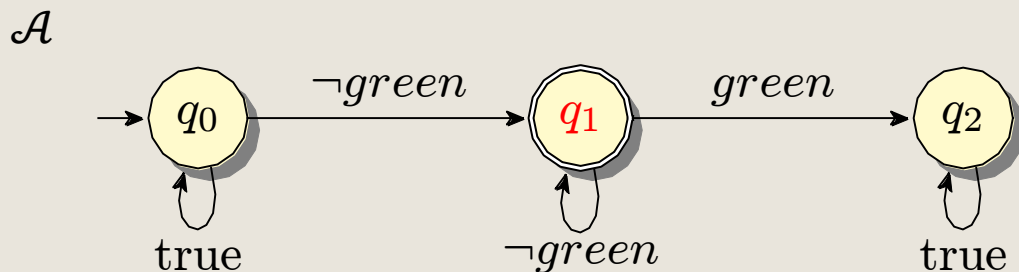
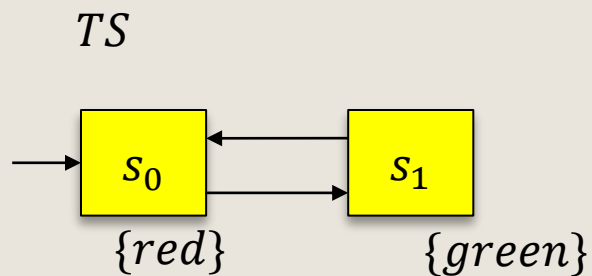
Let $q_0, q_1, \dots$ be an accepting run in $\mathcal{A}$ for $\text{trace}(\pi)$

Then $q_0 \in Q_0$ and $q_i \xrightarrow{L(s_i)} q_{i+1}$ for all $i \geq 0$ and $\exists^\infty .q_i \in F$

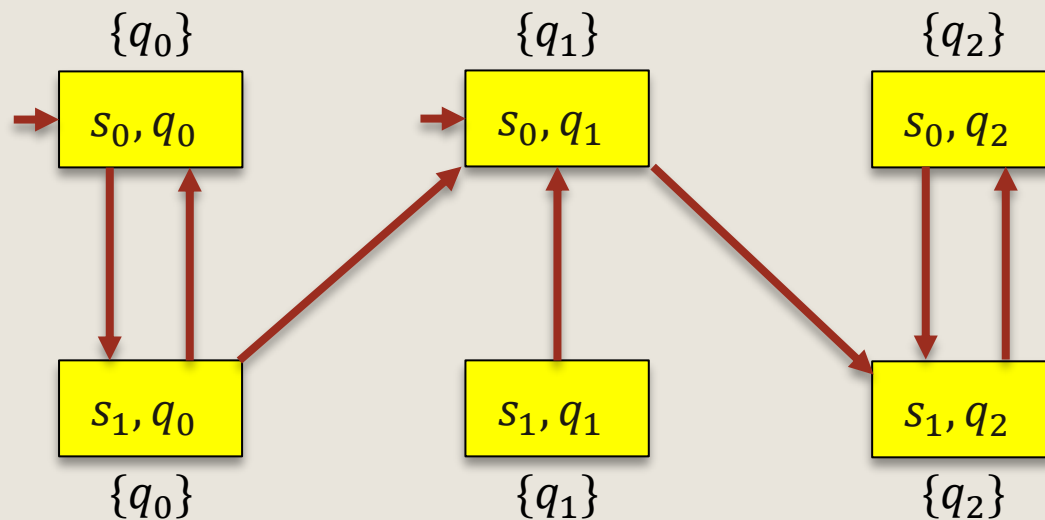
$\pi' = \langle s_0, q_1 \rangle, \langle s_1, q_2 \rangle, \dots \in \text{Paths}(TS \times \mathcal{A})$ such that $\pi' \notin P_{\text{pers}(\mathcal{A})}$

דוגמה: אינסוף פעמים ירוק?

אוטומט למילים שאינן מקיימות את התכונה



$TS \times \mathcal{A}$

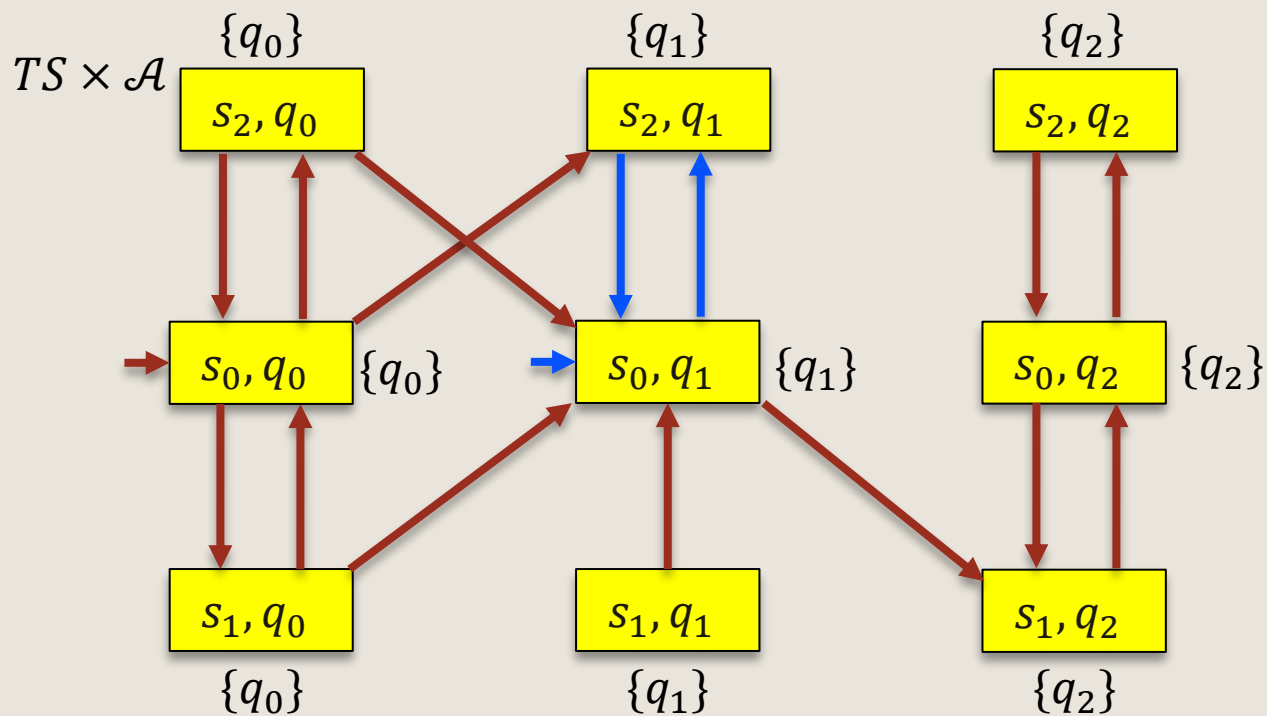
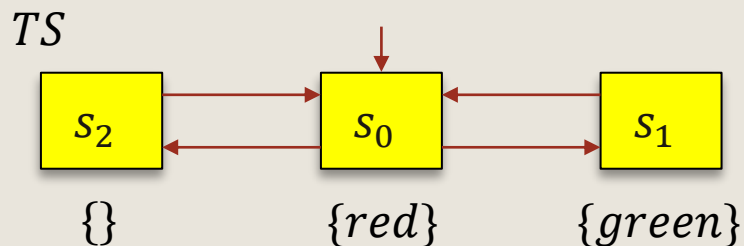
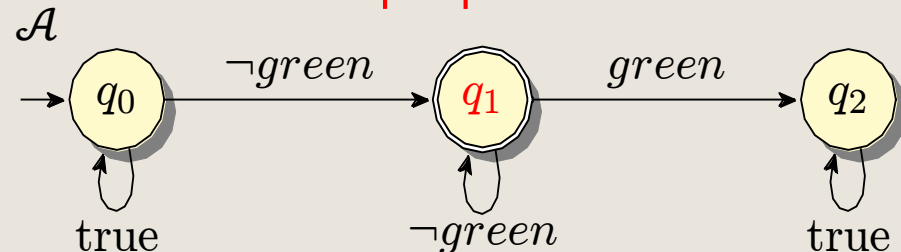


אין מעגל המכיל מצב המתויג ב- q_1

דוגמה: אינסוף פעמים ירוק?



אוטומט למילים שאינן מקיימות את התכונה



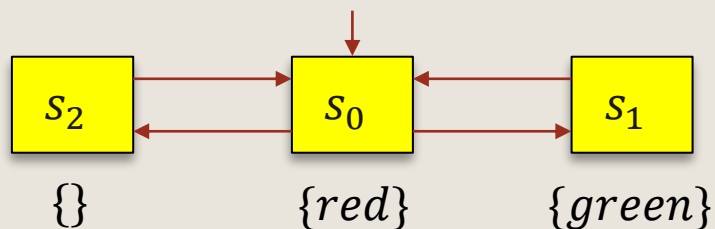
יש מעגל נגיש המכיל מצב המתויג ב- q_1

למה לא להשתמש באוטומט למילים המקיימות את התכונה?
 דוגמה: לאחר זמן סופי מפסיקים להדליק ירוק

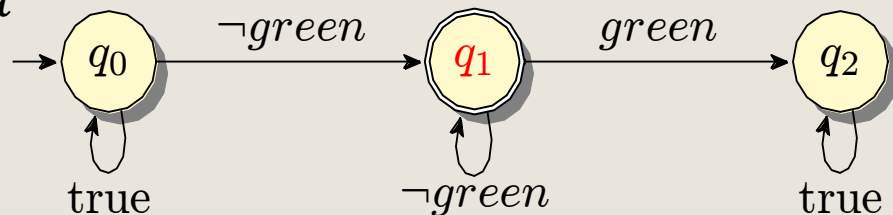


אוטומט למילים המקיימות את התכונה

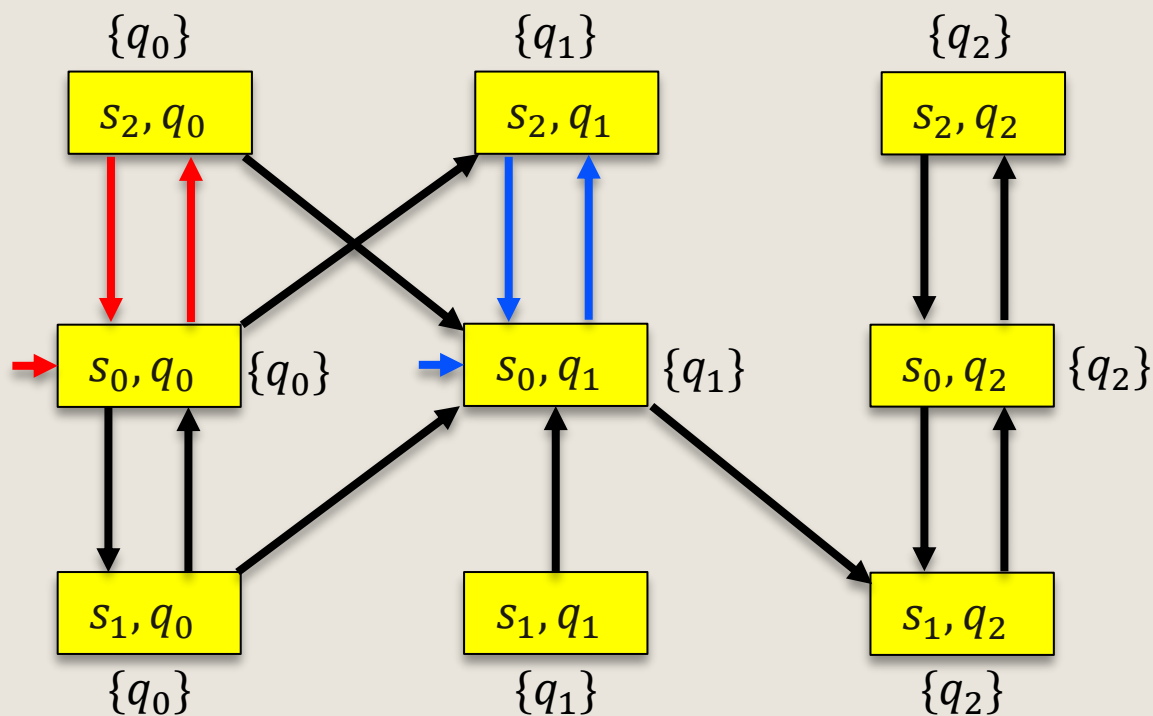
TS



$\mathcal{A}$



$TS \times \mathcal{A}$



קיום ריצה
 שאינה
 מקבלת לא
 אומר שיש
 "באג"



בדיקת התמדה וזיהוי מעגלים

עבור:

- מערכת מצבים TS מעל AP בלי מצבים סופניים
- ביטוי Φ בתחשיב פסוקים מעל AP (פסוק מצב)

$$TS \models \text{Eventually}(\text{Always}(\Phi))$$

אם ורק אם

$$\nexists s \in \text{Reach}(TS) . s \neq \Phi \wedge s \in \text{Post}^*(s)$$



ראשי פרקים

בדיקה תכונות בטיחות רגולריות



בדיקת תכונות! -רגולריות

בדיקת תכונות התמדה

רדוקציה לתכונות התמדה

תכונות התמדה (persistence)



חיפוש DFS מקונן



סיכום תכונות רגולריות



זיהוי מעגלים

איך נבדוק אם קיים מעגל נגיש המכיל מצב- $\neg\Phi$?

- אפשרות ראשונה:

- חישוב רכיבי הקשירות החזקים בגרף $G(TS)$
- בדיקה אם קיים רכיב קשירות חזק לא טריוויאלי נגיש ממצב התחלתי
- ומכיל מצב $\neg\Phi$
- התכונה "לבסוף תמיד Φ " נפסלת אם ורק אם נמצא כזה רכיב

- אפשרות שנייה:

- שימוש באלגוריתם DFS מקונן
- מתאים יותר לאימות on-the-fly
- קל יותר לייצר דוגמאות נגדיות



DFS דו-שלבי

1. מצא את כל מצבי ה- $\neg\Phi$ הנגישים ממצב התחלתי כלשהו
 - מתבצע ע"י חיפוש DFS רגיל
2. עבור כל מצב $\neg\Phi$ נגיש, בדוק אם הוא שייך למעגל
 - התחל חיפוש DFS מ- s
 - לכל מצב נגיש מ- s , בדוק האם יש קשת "חוזרת" ל- s
- סיבוכיות זמן: $\mathcal{O}(N \cdot (|\Phi| + N + M))$
 - באשר N הוא מספר המצבים ו- M מספר המעברים
 - חלקים הנגישים דרך K מצבי $\neg\Phi$ ייסרקו K פעמים



ראשי פרקים

בדיקה תכונות בטיחות רגולריות

בדיקת תכונות ω -רגולריות

בדיקת תכונות התמדה

רדוקציה לתכונות התמדה

תכונות התמדה (persistence)

חיפוש DFS מקונן

סיכום תכונות רגולריות



סיכום תכונות רגולריות (1)

- שפות המתקבלות ע"י NFA/DFA = שפות רגולריות
- משמשות לתיאור רישות רעות של תכונות בטיחות רגולריות
- בדיקת תכונות בטיחות רגולריות = בדיקת שמורה על מכפלה
- "אף פעם לא לבקר מצב מקבל" של ה-DFA עבור הרישות הרעות
- מסתכם בפיתרון בעיית נגישות באמצעות DFS
- שפות ω -רגולריות הן קבוצות של מילים אינסופיות
- ניתנות לתיאור ע"י ביטויים ω -רגולריים
- שפות המתקבלות ע"י NBA = שפות ω -רגולריות
- משמשות לתיאור תכונות ω -רגולריות



סיכום תכונות רגולריות (2)

- DBA הם בעלי יכולת תיאור פחותה מאשר NBA
- לדוגמה: אי אפשר לתאר תכונות התמדה כמו "בסופו של דבר תמיד a "
- GNBA דורשים אינסוף ביקורים בכל אחת מהקבוצות המקבלות
- השפות המתקבלות ע"י GNBA = שפות ω -רגולריות
- בדיקת תכונה ω -רגולרית = בדיקת תכונת התמדה על מכפלה
- ל-NBA של המשלים: "בסופו של דבר, אף פעם לא מצב מקבל"
- תכונות התמדה ניתנות לבדיקה בזמן ליניארי באמצעות DFS מקונן
- DFS מקונן = DFS עבור מצבי Φ – נגשים + DFS לזיהוי מעגלים